T3000 API & Protocol

Best Practices · built Nov 13, 2026

Best Practices

Recommended practices for optimal T3000 system operation.

System Design

Network Architecture

- Use dedicated VLAN for BACnet devices
- Assign static IP addresses
- Document network topology
- Implement firewall rules

Device Configuration

- Use consistent naming conventions
- Document all settings
- Export configurations regularly
- Maintain firmware versions

Data Management

- Set appropriate polling rates
- Archive old trend data
- Monitor database size
- Regular backups

Operations

Daily Tasks

- Check alarm status
- Review active overrides
- Monitor system health
- Verify auto-refresh working

Weekly Tasks

- Review trend data
- Check communication logs
- Update schedules as needed
- Test alarm notifications

Monthly Tasks

- Export configuration backups
- Review and optimize settings
- Update documentation
- Check disk space

Security

- Change default passwords
- Limit user access
- Use HTTPS in production
- Keep software updated
- Monitor access logs

Best Practices for T3000 Development

Code Architecture

Modular Design

TYPESCRIPT

```typescript
// ✅ Good: Separation of concerns
class DeviceManager {
  constructor(
    private connection: ConnectionPool,
    private cache: CacheManager,
    private logger: Logger
  ) {}

  async readPoint(deviceId: number, point: string): Promise<number> {
    // Check cache first
    const cached = this.cache.get(`${deviceId}:${point}`);
    if (cached) return cached;

    // Read from device
    const value = await this.connection.read(deviceId, point);

    // Update cache
    this.cache.set(`${deviceId}:${point}`, value, 10000);

    return value;
  }
}

// ❌ Bad: Tightly coupled
class DeviceManager {
  async readPoint(deviceId: number, point: string): Promise<number> {
    const cachedValue = localStorage.getItem(`${deviceId}:${point}`);
    if (cachedValue) return parseFloat(cachedValue);

    const response = await fetch(`http://localhost:9103/api/devices/${deviceId}/points/$
    const value = (await response.json()).value;

    localStorage.setItem(`${deviceId}:${point}`, value.toString());
    return value;
```

```
  }
}
```

Dependency Injection

TYPESCRIPT

```typescript
// Service container
class ServiceContainer {
  private services = new Map<string, any>();

  register<T>(name: string, factory: () => T): void {
    this.services.set(name, factory);
  }

  get<T>(name: string): T {
    const factory = this.services.get(name);
    if (!factory) throw new Error(`Service ${name} not registered`);
    return factory();
  }
}

const container = new ServiceContainer();

// Register services
container.register('logger', () => new ConsoleLogger());
container.register('cache', () => new MemoryCache());
container.register('connection', () =>
  new ConnectionPool(container.get('logger'))
);

// Use services
const deviceManager = new DeviceManager(
  container.get('connection'),
  container.get('cache'),
  container.get('logger')
);
```

Error Handling

Comprehensive Error Types

TYPESCRIPT

```typescript
enum ErrorCode {
  NETWORK_ERROR = 'NETWORK_ERROR',
  DEVICE_OFFLINE = 'DEVICE_OFFLINE',
  TIMEOUT = 'TIMEOUT',
  INVALID_DATA = 'INVALID_DATA',
  PERMISSION_DENIED = 'PERMISSION_DENIED'
}

class T3000Error extends Error {
  constructor(
    public code: ErrorCode,
    message: string,
    public details?: any
  ) {
    super(message);
    this.name = 'T3000Error';
  }
}

// Usage
async function readDevicePoint(deviceId: number, point: string): Promise<number> {
  try {
    const response = await fetch(`/api/devices/${deviceId}/points/${point}`);

    if (!response.ok) {
      if (response.status === 404) {
        throw new T3000Error(
          ErrorCode.DEVICE_OFFLINE,
          `Device ${deviceId} not found`,
          { deviceId }
        );
      }
      throw new T3000Error(
        ErrorCode.NETWORK_ERROR,
        `HTTP ${response.status}: ${response.statusText}`
      );
```

```
    }

    return (await response.json()).value;
  } catch (error) {
    if (error instanceof T3000Error) throw error;

    throw new T3000Error(
      ErrorCode.NETWORK_ERROR,
      'Failed to read device point',
      { originalError: error }
    );
  }
}
```

Retry Strategies

TYPESCRIPT

```typescript
interface RetryConfig {
  maxRetries: number;
  initialDelay: number;
  maxDelay: number;
  backoffFactor: number;
}

async function withRetry<T>(
  operation: () => Promise<T>,
  config: RetryConfig = {
    maxRetries: 3,
    initialDelay: 100,
    maxDelay: 5000,
    backoffFactor: 2
  }
): Promise<T> {
  let lastError: Error;
  let delay = config.initialDelay;

  for (let attempt = 0; attempt <= config.maxRetries; attempt++) {
    try {
      return await operation();
    } catch (error) {
      lastError = error as Error;

      // Don't retry on certain errors
      if (error instanceof T3000Error &&
          error.code === ErrorCode.PERMISSION_DENIED) {
        throw error;
      }

      if (attempt < config.maxRetries) {
        await new Promise(resolve => setTimeout(resolve, delay));
        delay = Math.min(delay * config.backoffFactor, config.maxDelay);
      }
    }
  }
```

```
  throw lastError!;
}
```

Performance Best Practices

Efficient Data Fetching

TYPESCRIPT

```typescript
// ✅ Good: Batch requests
async function readMultiplePoints(
  deviceId: number,
  points: string[]
): Promise<Map<string, number>> {
  const response = await fetch(
    `/api/devices/${deviceId}/points?points=${points.join(',')}`
  );

  const data = await response.json();
  return new Map(Object.entries(data));
}

// ❌ Bad: Individual requests
async function readMultiplePoints(
  deviceId: number,
  points: string[]
): Promise<Map<string, number>> {
  const values = new Map<string, number>();

  for (const point of points) {
    const response = await fetch(
      `/api/devices/${deviceId}/points/${point}`
    );
    const data = await response.json();
    values.set(point, data.value);
  }

  return values;
}
```

Memoization

TYPESCRIPT

```typescript
function memoize<T extends (...args: any[]) => any>(
  fn: T,
  ttl: number = 5000
): T {
  const cache = new Map<string, { value: any; expires: number }>();

  return ((...args: any[]) => {
    const key = JSON.stringify(args);
    const cached = cache.get(key);

    if (cached && cached.expires > Date.now()) {
      return cached.value;
    }

    const value = fn(...args);
    cache.set(key, { value, expires: Date.now() + ttl });

    return value;
  }) as T;
}

// Usage
const getDeviceInfo = memoize(
  async (deviceId: number) => {
    const response = await fetch(`/api/devices/${deviceId}`);
    return response.json();
  },
  60000  // Cache for 1 minute
);
```

Testing

Unit Testing

TYPESCRIPT

```typescript
import { describe, it, expect, vi } from 'vitest';

describe('DeviceManager', () => {
  it('should read point from cache if available', async () => {
    const mockConnection = {
      read: vi.fn()
    };

    const mockCache = {
      get: vi.fn(() => 72.5),
      set: vi.fn()
    };

    const manager = new DeviceManager(
      mockConnection as any,
      mockCache as any,
      console
    );

    const value = await manager.readPoint(389001, 'IN1');

    expect(value).toBe(72.5);
    expect(mockCache.get).toHaveBeenCalledWith('389001:IN1');
    expect(mockConnection.read).not.toHaveBeenCalled();
  });

  it('should fetch from device and update cache on miss', async () => {
    const mockConnection = {
      read: vi.fn(() => Promise.resolve(75.0))
    };

    const mockCache = {
      get: vi.fn(() => null),
      set: vi.fn()
    };
```

```
    const manager = new DeviceManager(
      mockConnection as any,
      mockCache as any,
      console
    );

    const value = await manager.readPoint(389001, 'IN1');

    expect(value).toBe(75.0);
    expect(mockConnection.read).toHaveBeenCalledWith(389001, 'IN1');
    expect(mockCache.set).toHaveBeenCalledWith('389001:IN1', 75.0, 10000);
  });
});
```

Integration Testing

TYPESCRIPT

```
import { test, expect } from 'vitest';

test('full device data flow', async () => {
  // Start test server
  const server = await startTestServer();

  // Add test device
  await server.addDevice({
    id: 389001,
    name: 'Test Device',
    ip: '127.0.0.1'
  });

  // Read point
  const client = new T3000Client('http://localhost:9103');
  const value = await client.readPoint(389001, 'IN1');

  expect(value).toBeGreaterThanOrEqual(0);

  await server.stop();
});
```

Security Best Practices

Input Validation

TYPESCRIPT

```typescript
import { z } from 'zod';

// Define schema
const WritePointSchema = z.object({
  deviceId: z.number().int().positive(),
  point: z.string().regex(/^(IN|OUT|VAR)\d{1,2}$/),
  value: z.number().min(-100).max(200),
  priority: z.number().int().min(1).max(16).optional()
});

// Validate input
app.post('/api/write-point', async (req, res) => {
  try {
    const data = WritePointSchema.parse(req.body);

    await writePoint(data.deviceId, data.point, data.value, data.priority);

    res.json({ success: true });
  } catch (error) {
    if (error instanceof z.ZodError) {
      res.status(400).json({ error: error.errors });
    } else {
      res.status(500).json({ error: 'Internal error' });
    }
  }
});
```

Authentication & Authorization

TYPESCRIPT

```typescript
import jwt from 'jsonwebtoken';

interface UserPayload {
  userId: string;
  role: 'admin' | 'operator' | 'viewer';
}

function authenticateToken(req: Request, res: Response, next: NextFunction) {
  const token = req.headers.authorization?.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'No token provided' });
  }

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET!) as UserPayload;
    req.user = payload;
    next();
  } catch (error) {
    res.status(403).json({ error: 'Invalid token' });
  }
}

function requireRole(role: string) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (req.user?.role !== role && req.user?.role !== 'admin') {
      return res.status(403).json({ error: 'Insufficient permissions' });
    }
    next();
  };
}

// Usage
app.post('/api/write-point',
  authenticateToken,
  requireRole('operator'),
  async (req, res) => {
    // Handle write
```

```
  }
);
```

Documentation

Code Comments

TYPESCRIPT

```typescript
/**
 * Reads a point value from a device with automatic retry and caching
 *
 * @param deviceId - The unique device identifier
 * @param point - Point name (e.g., 'IN1', 'OUT5', 'VAR10')
 * @param options - Optional configuration
 * @returns The current point value
 * @throws {T3000Error} If device is offline or read fails after retries
 *
 * @example
 * ```typescript
 * const temp = await readPoint(389001, 'IN1');
 * console.log(`Temperature: ${temp}°F`);
 * ```
 */
async function readPoint(
  deviceId: number,
  point: string,
  options?: ReadOptions
): Promise<number> {
  // Implementation
}
```

Next Steps

- FAQ
- Troubleshooting
- Performance Tuning